Task 4 : Question 1 :

Convert each GPS reading into meters from the grid origin, then rotate those meters by the grid's offset angle. The rover's heading is never needed.

The grid is tiny, so I can treat the Earth as flat around the origin.

- North distance: $N = (lat - lat_0) \times (\pi/180) \times R$
- East distance: $E = (lon - lon_0) \times (\pi/180) \times R \times \cos(lat_0)$

Here, $R = 6{,}371{,}000$ m. Longitude degrees shrink towards the poles, which is why East needs the $\cos(lat_0)$ factor. Over a few meters this approximation is accurate to well under a millimeter, so the full Haversine formula isn't needed.

Also, I would place the rovers at two known points along one grid edge, such as the

origin and the next corner. I would record GPS at each, average many readings to reduce noise, and compute the bearing between them using the bearing formula. That ~~being~~ bearing is θ. the angle by which the grid's y-axis is rotated clockwise from true north.

$$x = E \cdot \cos\theta - N \cdot \sin\theta$$

$$y = E \cdot \sin\theta + N \cdot \cos\theta$$

I check this ~~wh~~ with a simple case. If θ is 0, the grid points exactly north. Then $x = E$ and $y = N$, which is correct.

I average a few reading because GPS jumps a little. I also check that the final point is inside the 2x2 area. If it is outside, something is wrong.

GPS tells me where the rover is, not which way it faces. Position only ~~d~~needs

the two steps above. Heading would only matter if I wanted to steer the rover.

GPS works with true north. A compass points to magnetic north, which is a bit different and changes from place to place. That would add another error on top of the grid tilt.

If the grid is only 2m wide, normal GPS is too weak. For a real result, I would use RTK GPS, which is accurate to a few centimeters. Or I would combines GPS with an IMU or wheel odometry using a kalman filter

Question 2

On Linux, USB serial devices get names like /dev/ttyUSB0 and /dev/ttyUSB1. The system gives these names in the order it finds the devices. The names do not belong to a specific sensor, and they can change after a reboot.

My teammate copied the GPS config for the IMU. If the port name stayed the same, both nodes open the same port. Two programs then read from one port and take bytes from each other. If the IMU node also sets a different baud rate, the GPS data turns into garbage. So the GPS looks silent.

It also explains the reboot problem. After reboot, the system may find the IMU first

then ttyUSB0 belongs to the IMU and now the GPS is the one that fails.

Each sensor works alone because nothing else is using the port.

-How would I fix it:

1. find the details of each device: I took for the vendor ID, product ID and serial number,

2. Write a udev rule that gives each device a fixed name.

3. Reload the rules.

4. change the GPS node to use and the IMU node to use. Each node should have its own port and baud rate.

5. If both adapters are the same type and have no serial number. This name depends on the physical USB port. I then label the ports on the rover.

Scanned with CamScanner